Recursive Towers of Hanoi on 1980s Educational Microcomputers

A Tale of Two Architectures, Forty-Two Years Late

LambdaMikel Claude (Opus 4.7, Anthropic)

May 2026

Abstract

We present two implementations of the recursive Towers of Hanoi algorithm targeting educational microcomputer systems from the early 1980s: the *Kosmos CP1* (1983, Intel 8049 based) and the *Busch Microtronic 2090* (1981, TI TMS1600 based). The CP1 implementation reads as a textbook example of recursion: the machine provides indirect addressing, indirect jumps, and enough memory to host a conventional call stack. The Microtronic implementation, however, must contend with a platform that offers none of those primitives—no stack, no indirect addressing, no indirect jumps, no random-access memory beyond the register file, and a four-bit data path. We show how recursion can nonetheless be realised on the Microtronic by exploiting its rarely-used dual register bank as a substrate for two hand-built stacks, and by replacing indirect returns with a small marker-dispatch table. We compare the two implementations as engineering artefacts and conclude with an account of physically embodying the algorithm: each host drives an Arduino over a simple four-bit protocol, realised here as a live 64×32 LED-matrix animation of the solve, with a pan/tilt robot arm as the original target.

1 Introduction

The Towers of Hanoi is the textbook example of tree recursion. Its implementation in any modern programming language is unremarkable; the language's runtime stack handles the recursive descent and the problem reduces to a six-line function. On a machine that has no hardware stack, no indirect addressing, and only sixteen four-bit registers, the story is rather different.

This article concerns two such machines, both sold as educational microcomputers in the early 1980s, and both essentially obsolete by 1990:

- the **Kosmos CP1** (1983), built around an Intel 8049 microcontroller, with hexadecimal keypad and seven-segment display, and (with the CP3 expansion) 256 words of memory;
- the **Busch Microtronic 2090** (1981), built around a TI TMS1600 calculator-derived microcontroller, with the same physical interface but a fundamentally simpler architecture—no addressable memory other than the register file, and no stack of any kind.

Both machines were sold to teach young people how computers "really work," and both succeed at this in ways the marketing material did not anticipate. The CP1 teaches you what *a* computer looks like. The Microtronic teaches you what a computer might be made to do when the architecture refuses to help.

A clarification worth making at the outset, because the rest of the article would otherwise be misleading: on neither machine does the user program the underlying microcontroller directly. The Intel 8049 inside the CP1 and the TI TMS1600 inside the Microtronic each run a fixed firmware program—an interpreter—that implements a higher-level instruction set of the manufacturer's own design. The programs entered through the hexadecimal keypad are sequences of these higher-level operations; the 8049 or TMS1600 native machine code beneath them is frozen in

ROM and uninvolved in any program the user writes. When we refer in what follows to "the CP1's instruction set" or "the Microtronic's opcodes," it is always the *interpreter's* instruction set that is meant; the underlying silicon ISAs play no role in the discussion. In effect, each machine exposes its own small virtual CPU to the learner—a vastly more austere relative of the idea behind a Java Virtual Machine or a Smalltalk image, but the same trick at the foundation. All the architectural constraints discussed below—the missing indirect addressing, the un-nestable `CALL`/`RET`, the four-bit-wide registers, the inaccessible program counter—are constraints of the *interpreter*, not of the silicon it runs on.

We will spend the bulk of this article on the Microtronic implementation, because that is the more interesting story, and only briefly summarise the CP1 program, which is in essence the form one would write if given the choice of platform. We close with a short discussion of the recent extension of both programs to drive external hardware through an Arduino-mediated digital protocol—realised as a live LED-matrix animation of the solve, with a Hanoi-solving robot arm as the motivating goal—demonstrating that the original programs were really doing useful work, not merely simulating it on a seven-segment display.

The recursive procedure under discussion is, throughout, the following:

```
function MoveTower(disk, source, dest, spare):
    if disk == 1:
        emit_move(source, dest)
    else:
        MoveTower(disk - 1, source, spare, dest)
        emit_move(source, dest)
        MoveTower(disk - 1, spare, dest, source)
```

The challenge on either machine is implementing the two recursive calls without losing the caller's local state. On a stack machine this is free; on the CP1 it is a deliberate engineering exercise; on the Microtronic it is essentially a small piece of architecture research.

2 The Kosmos CP1

2.1 What the machine provides

The CP1 is a fairly orthodox 8-bit accumulator machine. With the CP3 memory expansion it offers 256 words of writable memory, addressable 0–255. Beyond the conventional load/store/arithmetic operations, it offers three primitives that turn out to be decisive for any recursive program:

- `LIA` (19.xxx): *load indirect*. Treats memory cell xxx as a pointer; loads the accumulator from the address that cell contains.
- `AIS` (20.xxx): *store indirect*. Stores the accumulator at the address pointed to by cell xxx.
- `SIU` (21.xxx): *jump indirect*. Transfers control to the address held in cell xxx.

With these three instructions, recursion is straightforward: one implements a stack as a region of memory, manages a stack pointer in a fixed cell, and uses `SIU` to return to the caller via a saved address.

2.2 Memory layout

The CP1 implementation maintains four stacks simultaneously: three value stacks (*source*, *spare*, *dest*) and one return stack. Rather than packing all four into a single interleaved frame layout, the program uses three *parallel* value stacks at stride three, plus a separate return stack:

```
 140  aux 1            150  const 1
 141  aux 2            151  const 3
 142  aux 3
 143  return slot 1
 144  return slot 2

 145  disk number
 146  *source          (stack pointer; initially 200)
 147  *spare           (stack pointer; initially 201)
 148  *dest            (stack pointer; initially 202)
 149  *return          (stack pointer; initially 240)

 200..239  value stacks  (13 frames, stride 3)
 240..255  return stack  (16 entries)
```

The three value-stack pointers begin at 200, 201, and 202. Pushing a frame is then a matter of adding three to each pointer, since the three values for the new frame land naturally at the next stride-three slot. The disk number is treated as a single mutable cell rather than being stacked; it is decremented on entry and incremented on return, which is correct because the algorithm only ever explores depths monotonically.

2.3 The subroutine-call convention

The CP1 has no hardware call instruction. The program implements one by hand: before any `SPU` (unconditional jump) used as a call, the caller writes the desired return address into a fixed memory cell; the callee returns by `SIU` through that cell. Two such cells exist, named *simple return 1* and *simple return 2*, which allows one level of subroutine nesting (the inner subroutine may itself call another, provided it uses the other slot). For deeper nesting—in particular, for recursion—the program pushes the return address onto the return stack instead, and the dedicated *return block* at address 136 pops the top of the return stack into *simple return 1* and `SIU`s through it.

In effect, the CP1 program reconstructs the runtime apparatus of a single-threaded recursive language: a call stack, a return mechanism, and a frame layout. None of it is exotic. The interesting choice is the *stride-3 parallel stacks*, which save the arithmetic of computing per-field offsets inside packed frames; once the three pointers are advanced together, an indirect load of, say, `*source` yields the source peg of the current frame without further work.

2.4 Code walkthrough

The full recursive-call site, prologue, and epilogue are short enough to read end-to-end. The first recursive call in `move_tower` performs `move_tower(disk-1, source, spare, dest)`; its code sits at cells 023–034:

```
023 ako 04.026   # load 26 (= label_0)
024 abs 06.143   # write into 'simple return 1'
025 spu 09.098   # call save_and_push
026 lda 05.140   # accu = saved source
027 ais 20.146   # store into new top of source stack
028 lda 05.142   # accu = saved dest
029 ais 20.148   # store into new top of dest stack ...
                 # ... but used as the new SPARE: dest → spare
030 lda 05.141   # accu = saved spare
031 ais 20.147   # store into new top of spare stack ...
                 # ... used as the new DEST: spare → dest
032 ako 04.035   # load label_2 address
033 ais 20.149   # push onto return stack
034 spu 09.020   # GOTO move_tower
```

Lines 028–031 are the swap that makes the recursion work: the new `dest` is the old `spare`, and the new `spare` is the old `dest`. The reads come from scratch cells 140/141/142 because `save_and_push` has already snapshotted the caller's frame there before advancing the pointers.

The prologue, `save_and_push`, occupies cells 098–119:

```
098 lda 05.145   # accu = n
099 sub 08.150   # n - 1
100 abs 06.145   # write back
101 lia 19.146   # accu = source (indirect load)
102 abs 06.140   #   ... save to scratch
103 lia 19.148   # accu = dest
104 abs 06.141   #   ... save to scratch
105 lia 19.147   # accu = spare
106 abs 06.142   #   ... save to scratch
107 lda 05.146   # advance *source by 3
108 add 07.151   #   (cell 151 holds 3)
109 abs 06.146
110 lda 05.148   # advance *dest by 3
111 add 07.151
112 abs 06.148
113 lda 05.147   # advance *spare by 3
114 add 07.151
115 abs 06.147
116 lda 05.149   # advance *return by 1
117 add 07.150   #   (cell 150 holds 1)
118 abs 06.149
119 siu 21.143   # return via 'simple return 1'
```

The pattern is: snapshot the live frame into scratch, advance every stack pointer, return. The call-site setup code, equipped with the snapshot and the new (advanced) pointers, can now permute the arguments and write them into the new top of each value stack without needing temporaries of its own.

The epilogue, `pop_and_restore`, at cells 120–135 is the mirror image: increment `n`, retreat all four stack pointers, return. The frames themselves are left untouched in memory; the restoration is purely a pointer reset, which is one of the small elegances the indirect-addressing primitives `LIA`/`AIS` buy on this machine.

Finally, the *return block* at cells 136–138 does the indirect jump that the Microtronic cannot:

```
136 lia 19.149   # accu = top-of-return-stack (the saved label)
137 abs 06.143   # write into 'simple return 1'
138 siu 21.143   # indirect jump through 143 -- lands at the label
```

Cell 143 is the same *simple return 1* slot that `save_and_push` also returns through; the return block overwrites it with the popped address before jumping. The `SIU` instruction is the irreducible primitive the Microtronic lacks, and is what lets the CP1 implementation look, overall, like a textbook recursive program.

2.5 Capacity

The CP1 program supports up to thirteen disks within the default memory layout. With minor adjustments to stack base addresses (the comments suggest moving the value stack start to 170 and the return stack to 230), it can be pushed further still. The actual ceiling is the available memory.

3 The Busch Microtronic 2090

3.1 What the machine does not provide

The Microtronic is a different proposition. Its architecture omits nearly every primitive that makes the CP1 program straightforward:

- There is no addressable memory beyond the register file. All state must live in registers.
- There are only sixteen registers, each four bits wide.
- There is no indirect addressing; all instructions encode their operands as register or constant nibbles.
- There is no indirect jump; control transfer is by absolute eight-bit address only.
- There *is* a hardware `CALL`/`RET` pair, but it relies on a single return-address register that is neither user-accessible nor nestable. A second `CALL` issued while a return address is still live overwrites that register; the outer return address is then lost, and the subsequent `RET` jumps to the inner call site. The hardware `CALL`/`RET` mechanism is therefore confined to *leaf* subroutines—it cannot, by itself, support recursion.
- There is, however, a curiosity that turns out to matter enormously: *the register file is actually two banks of sixteen registers each*, with three exchange instructions (`EXRL`, `EXRM`, `EXRA`) that swap the contents of the low half, the high half, or the whole file with a hidden second bank.

For most of the Microtronic's history, the second bank was used—if at all—as a scratch area for temporarily preserving values across some calculation. The implementation discussed here uses it for something more structural.

3.2 The central idea

The implementation treats the Microtronic's two register banks as a two-page memory system. One bank holds the current active frame and the return stack; the other bank holds the value stack. The exchange instructions `EXRL` and `EXRM`, applied together, swap the two banks in their entirety; applied around a block of register-shuffling code, they let one bank be mutated while the other is left untouched and then "swapped back into focus."

Concretely:

- **Bank A** (the bank active at procedure entry): registers $\texttt{R}_8$ to $\texttt{R}_\texttt{F}$ hold the current frame and a return marker; registers $\texttt{R}_0$ to $\texttt{R}_7$ are left alone.
- **Bank B** (accessed only via `EXRL`+`EXRM` pairs): all sixteen registers $\texttt{R}_0$ to $\texttt{R}_\texttt{F}$ hold the value stack as a deque of up to four four-value frames.

The shift routines that implement push and pop on the value stack *themselves* bracket their work with `EXRL`+`EXRM` / `EXRL`+`EXRM`, so that from the caller's perspective the shifts appear atomic and the caller's own bank is restored on return. This bracketing pattern is what makes the whole architecture composable, and it is what allows the protocol code we describe in Section 5 to use the otherwise-unused registers $\texttt{R}_0$, $\texttt{R}_1$, $\texttt{R}_7$ as scratch and persistent state without disturbing either stack.

3.3 Frame layout and the push

Each call frame consists of four values—disk number, source peg, destination peg, spare peg—occupying four consecutive registers. Pushing a frame requires:

1. Calling the *value-stack shift-down* routine four times, each call shifting every register one position towards the lower indices (with the lowest register lost). After four shifts, four register slots have been freed at the top of the stack.
2. Switching to bank B (via `EXRM`) and writing the four new frame values into the freed slots.
3. Switching back to bank A, recording a small integer *return marker* in R_F to identify the call site, and unconditionally jumping (`GOTO`) to the recursive entry point.

The shift routine itself is unrolled and minimal:

```
70: f0d   # EXRL: swap low half of register file
71: f0e   # EXRM: swap high half
72: 010   # r0 := r1   (operand order: ORS means r{S} := r{R})
73: 021   # r1 := r2
74: 032   # r2 := r3
75: 043   # r3 := r4
76: 054   # r4 := r5
77: 065   # r5 := r6
78: 076   # r6 := r7
79: 087   # r7 := r8
7a: 098   # r8 := r9
7b: 0a9   # r9 := rA
7c: 0ba   # rA := rB
7d: 0cb   # rB := rC
7e: 0dc   # rC := rD
7f: 0ed   # rD := rE
80: 0fe   # rE := rF   (rF is the only register not written; it duplicates into rE)
81: f0e   # EXRM (back)
82: f0d   # EXRL (back)
83: f07   # RET
```

The two `EXR*` pairs are the magic. Inside the bracket the machine is operating on bank B, where the value stack lives; outside, on bank A, where the caller's working registers are. A single call to this routine shifts the entire bank-B register file by one position *without* touching bank A. Four calls in a row create the four-slot hole that a push requires.

A complete recursive call site brings the pieces together. The listing below is the program's first recursive call, performing `MoveTower(n-1, source, spare, dest)`:

```
15: f0e   # EXRM (to bank A, caller-visible side)
16: b70   # CALL value-stack shift-down (1 of 4)
17: b70   # ...
18: b70   # ...
19: b70   # 4 shifts: four-slot hole opens at top of bank B
1a: f0e   # EXRM (to bank B, where the new frame goes)
1b: 0bf   # MOV RB→RF: new n = caller's n
1c: 71f   # SUBI 1, RF: n := n - 1
1d: 0ae   # MOV RA→RE: new source = caller's source
1e: 09c   # MOV R9→RC: new spare  = caller's dest
1f: 08d   # MOV R8→RD: new dest   = caller's spare
20: f0e   # EXRM (back to bank A)
21: b50   # CALL return-stack shift-down: room for a new marker
22: 12f   # MOVI 2, RF: push return marker = 2
23: c0d   # GOTO MoveTower entry
```

Two points are worth dwelling on. First, the four `CALL`s at \$16–\$19 are leaf calls—they use the hardware return register, never nest into one another, and so the single-level `CALL`/`RET`

mechanism noted earlier is no obstacle. The recursive call itself, at $23, is deliberately *not* a `CALL` but a plain `GOTO`; the would-be return information is recorded in the software return stack at $21–$22, not in the hardware return register, which is left undisturbed.

Second, the new frame's `source`, `dest`, and `spare` fields are read from `R8`, `R9`, and `RA` on lines $1d–$1f, not from the `RC`–`RE` positions where they originally lived. The four shift-down calls have relocated them four positions toward the lower indices, and the setup code knows this—it reads them in their post-shift positions and writes them into the new top frame, performing the *dest* ↔ *spare* swap that the recursive call requires. The second recursive call, at $2e–$3b, follows the same shape with the opposite permutation and a different return marker.

3.4 The return mechanism

The hardware `CALL`/`RET` pair cannot nest, and the Microtronic has neither a user-accessible program counter nor an indirect jump. So even if the program were willing to save the return address by hand, there is nowhere to save it *from* and no way to jump back to it *through*. After a recursive call returns, control must somehow find its way back to one of three possible continuation sites in the body of `MoveTower`: the point after the first recursive call, the point after the second, or the top-level caller in main. Conventional return-address pushing is unworkable on all three counts.

The implementation solves this with a tiny *return marker*: each call pushes a small integer onto a one-register-deep return stack (in fact several registers deep, allowing the call chain to nest as deeply as the algorithm requires). After the call returns, control transfers to a fixed *dispatch block*:

```
b0: 91f   # CMPI 1, RF        (marker == 1?)
b1: E0b   # BRZ to 0b         (return to main)
b2: 92f   # CMPI 2, RF        (marker == 2?)
b3: E24   # BRZ to 24         (return to inner site 2)
b4: 93f   # CMPI 3, RF        (marker == 3?)
b5: E3c   # BRZ to 3c         (return to inner site 3)
b6: F00   # HALT              (error: unknown marker)
```

With only three distinct call sites in the program, three markers suffice. The dispatch costs a handful of compare-and-branch instructions per return, which is cheap relative to the sixty-instruction cost of pushing and popping a value-stack frame.

At the chosen return target, the caller's frame is reinstated by running the push sequence in reverse. After the dispatch at $b3 sends control to $24:

```
24: b60   # CALL return-stack shift-up: pop the consumed marker
25: b90   # CALL value-stack shift-up (1 of 4)
26: b90   # ...
27: b90   # ...
28: b90   # 4 shifts: caller's frame back at RC..RF in bank B
```

After $28, the caller's `n`, `source`, `dest` and `spare` are once again the top frame of the value stack, and execution resumes with the caller none the wiser.

3.5 Capacity

The Microtronic implementation supports up to four disks. The constraint is the value stack: each frame is four registers wide, and the deepest sensible level holds four frames simultaneously, exactly filling bank B's sixteen registers. A more aggressive packing—two two-bit peg labels and a two-bit disk number into a single register—would extend this to seven or eight disks, at the cost of significant additional pack/unpack code.

4 Comparison

The two implementations solve the same problem on machines of broadly similar physical scale (256 words of program memory, six-digit displays, hex keypads, single-digit kilohertz clock speeds), but they read as if they belonged to different decades.

	Kosmos CP1	**Busch Microtronic 2090**
CPU	Intel 8049 (8-bit)	TI TMS1600 (4-bit interpreted)
Memory beyond regs	256 words (with CP3)	none
Indirect load/store	`LIA`, `AIS`	not available
Indirect jump	`SIU`	not available
Bitwise AND	`UND`	not available
Number of registers	8-bit accumulator	16 four-bit registers, two banks
Stack realisation	memory region + pointer cell	shifts across the second bank
Subroutine return	indirect via memory cell	marker dispatch table
Disk-count ceiling	13 (default), more with effort	4
Per-move overhead	a few dozen instructions	roughly sixty (mostly shifts)
Lines of source	∼140	∼160
Reads like	a recursive program	a small machine you built

The CP1 implementation is *applied* computer science: it is what one writes when the platform provides the primitives the algorithm needs. The Microtronic implementation is *experimental* computer science: it is what one writes when those primitives must first be invented out of the available substrate. The bank-swap-as-stack idiom is the central piece of invention. It would generalise to other recursive algorithms—tree walks, mutual recursion, even simple interpreters—on the same hardware, but to the author's knowledge it had not appeared in print, in forty-two years of the Microtronic's existence, until this implementation was written in early 2024.

5 Physical Embodiment

Both programs have been extended to drive external hardware over a deliberately minimal digital protocol: the vintage host emits a stream of high-level $\langle source, dest \rangle$ move commands and an Arduino does everything else. The original target was a Towers-of-Hanoi-solving robot—a two-degree-of-freedom pan/tilt arm with an electromagnet at the tip, the Arduino handling all servo motion planning, magnet pulse-width control, and disk-position tracking. That arm is still under construction; the first embodiment actually *realised* drives a different actuator over the very same protocol—a 64×32 RGB LED matrix on which the disks slide between three pegs, rainbow-coloured by size (Figure 1). The vintage host still does the real work—the recursion and the optimal move sequence—while a second Arduino merely renders each move it is handed.

The communication protocol is deliberately minimal so that it can fit inside either machine's tiny I/O envelope. Four output lines from the host carry a three-bit move code (1–6, one per legal source-to-dest pair) and a strobe bit that toggles on every command; one input line returns to the host as a *busy* signal that is held high while the Arduino is in motion. The host's per-move logic is then:

1. Poll the busy line until low.
2. Compute the three-bit move code and toggle the strobe bit.
3. Atomically present all four output bits.
4. Poll the busy line until high (acknowledgement).

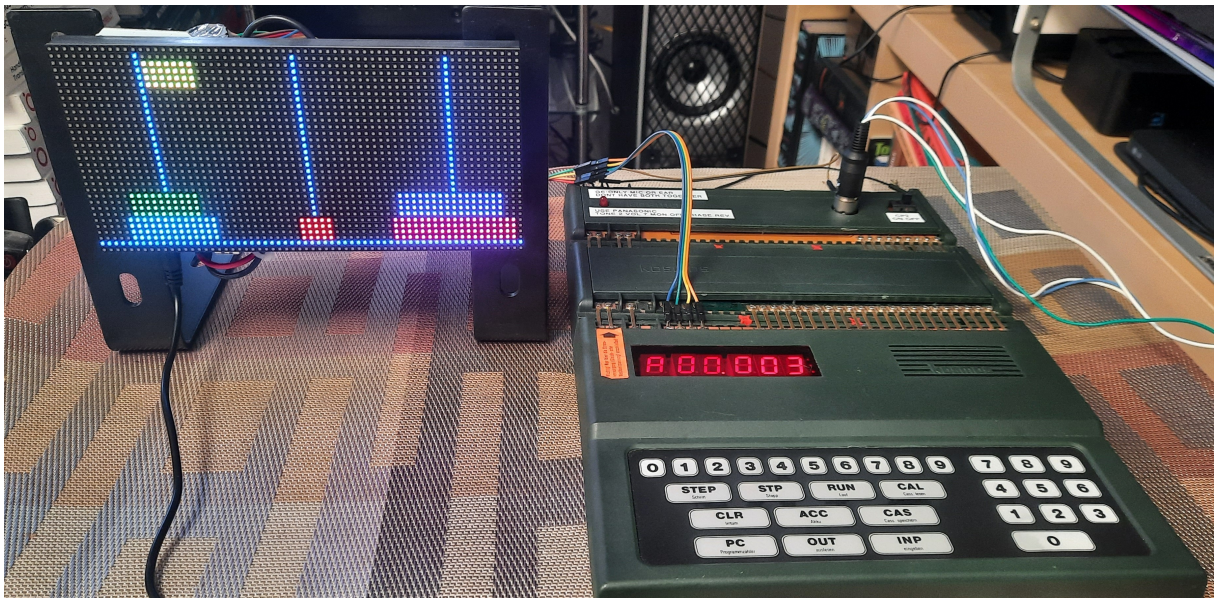


Figure 1: The realised embodiment: a Kosmos CP1 (right) driving a 64×32 RGB LED matrix (left) through the four-bit protocol, mid-solve. The CP1 runs the recursion and streams each move to an Arduino that animates the disks; the CP2 cassette module (top) loads the program. A full solve is online at `https://youtu.be/CkCNBzWRjq4`.

On the Microtronic, the four output bits are written by a single `DOT` instruction, which makes the protocol naturally race-free: the strobe transition and the data bits all change in the same instruction. On the CP1, the same effect is achieved by writing the three data bits first via three successive `P2A` instructions and the strobe bit last, so that when the Arduino's edge-triggered interrupt fires on the strobe, the data lines are already settled.

A subtle question on the Microtronic was where to store the single bit of persistent strobe state across recursive calls. The algorithm uses every register in both banks for the value and return stacks—or does it? In fact, registers $\mathtt{R}_0$ to $\mathtt{R}_7$ of the active (bank A) state are never written by the original recursive code. The shift routines do touch them, but *only* inside their `EXRL`-bracketed regions, where they operate on bank B's copies; the active-bank values are restored on exit. The strobe bit therefore lives in $\mathtt{R}_7$ of bank A and survives the full depth of the recursion without further machinery.

On the CP1 the equivalent state is simply a memory cell—though bringing the real hardware up turned an undocumented corner of the machine into a concrete bug. The CP1's `P2A` (Port-2 output) instruction latches *any* non-zero accumulator as a logic 1, not the least-significant bit; a strobe implemented as an incrementing counter therefore jammed high after the first move, and the receiver—correctly—never saw a second edge. Rewriting the toggle as $\mathit{strobe} := 1 - \mathit{strobe}$, so the emitted value is always exactly 0 or 1, fixed it. The episode is a reminder that these interpreters are documented only to the depth someone has so far needed.

6 Closing Remarks

The CP1 program is a finished engineering artefact: it does what recursion is supposed to do, on a machine that lets it. It is the program one would write today if one were trying to teach a student how recursion is implemented underneath a high-level language.

The Microtronic program is something different. It is a discovery about a forty-year-old machine—a demonstration that an architecture which appears to forbid recursion will, in fact, host it, provided the programmer is willing to construct the missing primitives by hand. The

bank-swap-as-stack idea is small, but it is the kind of thing that does not announce itself: in forty-two years of the Microtronic's existence, it had not been written down. It is in the nature of obscure platforms that they accumulate such secrets, and in the nature of long acquaintance that one occasionally notices them.

That the same machines can now drive live external hardware, through a fifty-cent serial protocol to a five-dollar microcontroller, is a fitting late chapter. The TMS1600 and the 8049 were designed to teach people what computers were; they are now, against considerable odds, animating a provably-optimal Towers-of-Hanoi solve on a bank of LEDs—and, once the arm is finished, moving real disks between three pegs in the same sequence. The teaching, it appears, was effective.

Authorship Note

The prose of this paper was generated by Claude Opus 4.7 (Anthropic), working from extended discussions of the source code with the human co-author. The two recursive Towers of Hanoi implementations themselves—the Microtronic 2090 program and the Kosmos CP1 program—were written by hand by the human co-author (LambdaMikel) and should be credited accordingly. The hardware-driving extensions to both programs (the Arduino sketches for the robot arm and the LED-matrix renderer, the patched `HANOI-ROBOT.MIC` and `HANOIC-CP1-ROBOT.txt`, and the shared four-bit GPIO protocol described in Section 5) were designed and written by Claude—the initial extension in the conversation that produced this paper, and the CP1 hardware bring-up and `P2A` fix in a later one.

A Source-Code Availability

The reference implementations for both machines, and the Arduino firmware for the robot arm and the LED-matrix renderer, are available at `https://github.com/lambdamikel/towers-of-hanoi`. The opcode reference used in decoding the Microtronic source is drawn from the emulator code at `https://github.com/lambdamikel/Busch-2090`; the Kosmos CP1 manual is available via the Internet Archive at `https://archive.org/details/cp-1-manual`. A standalone CP1 development toolchain—assembler, cassette-tape tools, and the emulator used to reproduce the `P2A` behaviour discussed in Section 5—is at `https://github.com/lambdamikel/kosmos-cp1-devel-toolchain`.